

GSOI 2026 暑期常规赛

第七试题解

Ehundategh & tfbz

Gioush 大队

2026 年 7 月

目录

T1 藿香正气液 (elixir)

T2 晨汐散余香 (aroma)

T3 灵舵渡千屿 (helm)

T4 沧溟探秘藏 (treasure)

吐槽时间

Contents

T1 藿香正气液 (elixir)

T2 晨汐散余香 (aroma)

T3 灵舵渡千屿 (helm)

T4 沧溟探秘藏 (treasure)

吐槽时间

数据点 1 ~ 5 25 Pts

- 首先，看到这一档只有 $n \leq 5 \times 10^3$ ，显然可以直接枚举 $1 \leq x < y \leq n$ ，再检查是否满足 $\gcd(x, y) = y - x$ 。

数据点 1 ~ 5 25 Pts

- ▶ 首先, 看到这一档只有 $n \leq 5 \times 10^3$, 显然可以直接枚举 $1 \leq x < y \leq n$, 再检查是否满足 $\gcd(x, y) = y - x$ 。
- ▶ 单次检查使用 Euclid 算法, 时间复杂度为 $\mathcal{O}(n^2 \log n)$, 空间复杂度为 $\mathcal{O}(1)$ 。这个做法的问题在于, 相同的差值被重复枚举了很多次。

数据点 6 ~ 10 50 Pts

- 根据上一档部分的启发，不妨固定差值 $d = y - x$ 。因为 $y = x + d$ ，所以

$$\gcd(x, y) = \gcd(x, x + d) = \gcd(x, d).$$

数据点 6 ~ 10 50 Pts

- 根据上一档部分分的启发，不妨固定差值 $d = y - x$ 。因为 $y = x + d$ ，所以

$$\gcd(x, y) = \gcd(x, x + d) = \gcd(x, d).$$

- 原条件等价于 $\gcd(x, d) = d$ ，也就是 $d \mid x$ 。于是可以令 $x = td$ 、 $y = (t + 1)d$ ，其中 $t \geq 1$ 。

数据点 6 ~ 10 50 Pts

- 根据上一档部分分的启发，不妨固定差值 $d = y - x$ 。因为 $y = x + d$ ，所以

$$\gcd(x, y) = \gcd(x, x + d) = \gcd(x, d).$$

- 原条件等价于 $\gcd(x, d) = d$ ，也就是 $d \mid x$ 。于是可以令 $x = td$ 、 $y = (t + 1)d$ ，其中 $t \geq 1$ 。
- 固定 d 后，合法的 t 共有 $\lfloor n/d \rfloor - 1$ 个，因此答案为

$$\sum_{d=1}^{\lfloor n/2 \rfloor} \left(\left\lfloor \frac{n}{d} \right\rfloor - 1 \right).$$

直接枚举 d 的时间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- 现在的瓶颈只剩下求和长度。可以发现， $\lfloor n/d \rfloor$ 只有 $\mathcal{O}(\sqrt{n})$ 种不同取值。

正解 100 Pts

- ▶ 现在的瓶颈只剩下求和长度。可以发现， $\lfloor n/d \rfloor$ 只有 $\mathcal{O}(\sqrt{n})$ 种不同取值。
- ▶ 若当前左端点为 L ，令 $q = \lfloor n/L \rfloor$ ，那么这一段相同整除值的右端点为

$$R = \min \left(\left\lfloor \frac{n}{2} \right\rfloor, \left\lfloor \frac{n}{q} \right\rfloor \right).$$

这一整段的贡献为 $(q-1)(R-L+1)$ 。

正解 100 Pts

- ▶ 现在的瓶颈只剩下求和长度。可以发现， $\lfloor n/d \rfloor$ 只有 $\mathcal{O}(\sqrt{n})$ 种不同取值。
- ▶ 若当前左端点为 L ，令 $q = \lfloor n/L \rfloor$ ，那么这一段相同整除值的右端点为

$$R = \min \left(\left\lfloor \frac{n}{2} \right\rfloor, \left\lfloor \frac{n}{q} \right\rfloor \right).$$

这一整段的贡献为 $(q-1)(R-L+1)$ 。

- ▶ 随后令 $L \leftarrow R+1$ 继续即可。时间复杂度为 $\mathcal{O}(\sqrt{n})$ ，空间复杂度为 $\mathcal{O}(1)$ 。

正解 100 Pts

- 对任意一对合法的 (x, y) , $d = y - x$ 被唯一确定, 并且 $d \mid x$, 所以它唯一对应一组满足 $(t+1)d \leq n$ 的正整数 (t, d) 。

正解 100 Pts

- ▶ 对任意一对合法的 (x, y) , $d = y - x$ 被唯一确定, 并且 $d \mid x$, 所以它唯一对应一组满足 $(t+1)d \leq n$ 的正整数 (t, d) 。
- ▶ 反过来, 每一组这样的 (t, d) 都给出 $x = td$ 、 $y = (t+1)d$, 且

$$\gcd(x, y) = d = y - x.$$

因此前面的求和既没有遗漏, 也没有重复。

正解 100 Pts

- ▶ 对任意一对合法的 (x, y) , $d = y - x$ 被唯一确定, 并且 $d \mid x$, 所以它唯一对应一组满足 $(t+1)d \leq n$ 的正整数 (t, d) 。
- ▶ 反过来, 每一组这样的 (t, d) 都给出 $x = td$ 、 $y = (t+1)d$, 且

$$\gcd(x, y) = d = y - x.$$

因此前面的求和既没有遗漏, 也没有重复。

- ▶ 答案可能超过 `int` 范围, 所有计数与乘法都需要使用 `long long`。

正解 100 Pts

```
1 long long n,Ans;
2 int main(){
3     freopen("elixir.in","r",stdin);
4     freopen("elixir.out","w",stdout);
5     scanf("%lld",&n);
6     long long Limit=n/2;
7     for(long long Left=1,Right;Left<=Limit;Left=Right+1){
8         long long Value=n/Left;
9         Right=min(Limit,n/Value);
10        Ans+=(Value-1)*(Right-Left+1);
11    }
12    printf("%lld\n",Ans);
13    return 0;
14 }
```

Contents

T1 藿香正气液 (elixir)

T2 晨汐散余香 (aroma)

T3 灵舵渡千屿 (helm)

T4 沧溟探秘藏 (treasure)

吐槽时间

数据点 1 ~ 4 20 Pts

- ▶ 首先枚举选出的香料集合。将这个集合的失效时间从小到大排序为 $d_1 \leq d_2 \leq \dots \leq d_s$ 。

数据点 1 ~ 4 20 Pts

- ▶ 首先枚举选出的香料集合。将这个集合的失效时间从小到大排序为 $d_1 \leq d_2 \leq \dots \leq d_s$ 。
- ▶ 这个集合存在合法安排，当且仅当对每个 i 都有 $d_i \geq i$ 。充分性只需按顺序放到第 $1, 2, \dots, s$ 天；必要性则来自前 i 份香料只能占用前 d_i 天。

数据点 1 ~ 4 20 Pts

- ▶ 首先枚举选出的香料集合。将这个集合的失效时间从小到大排序为 $d_1 \leq d_2 \leq \dots \leq d_s$ 。
- ▶ 这个集合存在合法安排，当且仅当对每个 i 都有 $d_i \geq i$ 。充分性只需按顺序放到第 $1, 2, \dots, s$ 天；必要性则来自前 i 份香料只能占用前 d_i 天。
- ▶ 对每个可行集合记录价值之和，再按集合大小更新答案。时间复杂度为 $\mathcal{O}(2^n n \log n + q)$ 。

数据点 5 ~ 8 40 Pts

- 这一档中所有 t_i 均相等，不妨记为 T 。因此，任意不超过 T 份香料都可以分别安排到不同的交易日。

数据点 5 ~ 8 40 Pts

- ▶ 这一档中所有 t_i 均相等，不妨记为 T 。因此，任意不超过 T 份香料都可以分别安排到不同的交易日。
- ▶ 于是只需要将价值从大到小排序并求前缀和。对于询问 p ，选择价值最大的 $\min(p, n, T)$ 份香料即可。

数据点 5 ~ 8 40 Pts

- ▶ 这一档中所有 t_i 均相等，不妨记为 T 。因此，任意不超过 T 份香料都可以分别安排到不同的交易日。
- ▶ 于是只需要将价值从大到小排序并求前缀和。对于询问 p ，选择价值最大的 $\min(p, n, T)$ 份香料即可。
- ▶ 时间复杂度为 $\mathcal{O}(n \log n + q)$ 。这里得到的启发是：若能够判断当前香料能否加入，就应当优先尝试价值更大的香料。

数据点 9 ~ 12 60 Pts

- ▶ 这一档中所有 $t_i \geq n$, 任意香料都可以放入前 n 个互不相同的交易日, 所以限制只剩下最多选择多少份。

数据点 9 ~ 12 60 Pts

- ▶ 这一档中所有 $t_i \geq n$ ，任意香料都可以放入前 n 个互不相同的交易日，所以限制只剩下最多选择多少份。
- ▶ 与上一档相同，将价值从大到小排序并求前缀和，第 p 次询问直接取前 $\min(p, n)$ 项。

数据点 9 ~ 12 60 Pts

- ▶ 这一档中所有 $t_i \geq n$ ，任意香料都可以放入前 n 个互不相同的交易日，所以限制只剩下最多选择多少份。
- ▶ 与上一档相同，将价值从大到小排序并求前缀和，第 p 次询问直接取前 $\min(p, n)$ 项。
- ▶ 两个特殊性质都说明了同一件事：答案应当是某个按价值降序选择序列的前缀，正解只需要解决一般情况下的可行性判断。

数据点 13 ~ 16 80 Pts

- 固定一次询问 p ，只需要考虑前 $h = \min(p, n)$ 个交易日。
按价值从大到小扫描香料，尝试把当前香料放到不晚于 $\min(t_i, h)$ 的最晚空闲日。

数据点 13 ~ 16 80 Pts

- ▶ 固定一次询问 p ，只需要考虑前 $h = \min(p, n)$ 个交易日。按价值从大到小扫描香料，尝试把当前香料放到不晚于 $\min(t_i, h)$ 的最晚空闲日。
- ▶ 若这个位置存在，就选择当前香料；否则跳过。用并查集维护每一天左侧最近的空闲日，便可以在近似常数时间内完成一次判断。

数据点 13 ~ 16 80 Pts

- ▶ 固定一次询问 p ，只需要考虑前 $h = \min(p, n)$ 个交易日。按价值从大到小扫描香料，尝试把当前香料放到不晚于 $\min(t_i, h)$ 的最晚空闲日。
- ▶ 若这个位置存在，就选择当前香料；否则跳过。用并查集维护每一天左侧最近的空闲日，便可以在近似常数时间内完成一次判断。
- ▶ 每次询问重新执行上述过程，时间复杂度为 $\mathcal{O}(n \log n + qn\alpha(n))$ ，可以通过这一档。

正解 100 Pts

- ▶ 前一档部分分对每个询问都重新做了一次贪心，但每次扫描香料的顺序完全相同，变化的只有最多选择多少份。

正解 100 Pts

- ▶ 前一档部分分对每个询问都重新做了一次贪心，但每次扫描香料的顺序完全相同，变化的只有最多选择多少份。
- ▶ 因此可以先不限制选择数量：将香料按收益从大到小排序，依次尝试把当前香料放在不晚于 t_i 的最晚空闲日。若存在这样的日期，就将它加入选择序列。

正解 100 Pts

- ▶ 前一档部分分对每个询问都重新做了一次贪心，但每次扫描香料的顺序完全相同，变化的只有最多选择多少份。
- ▶ 因此可以先不限制选择数量：将香料按收益从大到小排序，依次尝试把当前香料放在不晚于 t_i 的最晚空闲日。若存在这样的日期，就将它加入选择序列。
- ▶ 这里仍然选择最晚的空闲日，是因为这样不会占用更早的日期，给之后失效时间更早的香料留下了尽可能多的位置。

正解 100 Pts

- 为什么一次扫描能够回答所有询问？考虑任意一个可行的 s 元方案。扫描到其中收益最小的香料时，这个方案已经说明当前前缀至少能够选出 s 份香料。

正解 100 Pts

- ▶ 为什么一次扫描能够回答所有询问？考虑任意一个可行的 s 元方案。扫描到其中收益最小的香料时，这个方案已经说明当前前缀至少能够选出 s 份香料。
- ▶ 而上述贪心在每个收益前缀中都会选出尽可能多的香料：若还能够多选一份，就可以通过交换，把某份尚未选择的香料加入当前安排，与贪心无法加入它矛盾。

正解 100 Pts

- ▶ 为什么一次扫描能够回答所有询问？考虑任意一个可行的 s 元方案。扫描到其中收益最小的香料时，这个方案已经说明当前前缀至少能够选出 s 份香料。
- ▶ 而上述贪心在每个收益前缀中都会选出尽可能多的香料：若还能够多选一份，就可以通过交换，把某份尚未选择的香料加入当前安排，与贪心无法加入它矛盾。
- ▶ 所以贪心选出的第 s 份香料不会比任意可行 s 元方案中的最小收益更小，前 s 份就是恰好选择 s 份时的最优答案。证明只需要反复使用上述交换即可。

正解 100 Pts

- 实现时，把当前香料放在不晚于 t_i 的最晚空闲日。若这个位置为 d ，占用后令并查集中的 d 指向 $d-1$ 左侧最近的空闲日。

正解 100 Pts

- ▶ 实现时，把当前香料放在不晚于 t_i 的最晚空闲日。若这个位置为 d ，占用后令并查集中的 d 指向 $d-1$ 左侧最近的空闲日。
- ▶ 若查询结果为 0，就跳过当前香料；否则每次成功选择第 r 份香料时记录

$$\text{Ans}_r = \text{Ans}_{r-1} + w_i.$$

正解 100 Pts

- ▶ 实现时，把当前香料放在不晚于 t_i 的最晚空闲日。若这个位置为 d ，占用后令并查集中的 d 指向 $d-1$ 左侧最近的空闲日。
- ▶ 若查询结果为 0，就跳过当前香料；否则每次成功选择第 r 份香料时记录

$$\text{Ans}_r = \text{Ans}_{r-1} + w_i.$$

- ▶ 设最终选中 c 份香料，询问 p 的答案就是 $\text{Ans}_{\min(p,c)}$ 。总时间复杂度为 $\mathcal{O}(n \log n + q)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

```
1 struct item{
2     int Val,Limit;
3 }Line[MAXN];
4 bool cmp(item a,item b){return a.Val>b.Val;}
5 int n,q,Fa[MAXN],cnt=0,In1;
6 long long Ans[MAXN];
7 int Find(int x){return Fa[x]==x?x:Fa[x]=Find(Fa[x]);}
```

正解 100 Pts

```
1  sort(Line+1,Line+n+1,cmp);
2  for(int i=0;i<=n;i++) Fa[i]=i;
3  for(int i=1;i<=n;i++){
4      int Day=Find(min(Line[i].Limit,n));
5      if(Day==0) continue;
6      Fa[Day]=Find(Day-1);
7      cnt++;
8      Ans[cnt]=Ans[cnt-1]+Line[i].Val;
9  }
10 while(q-->0){
11     scanf("%d",&In1);
12     printf("%lld\n",Ans[min(In1,cnt)]);
13 }
```

正解 100 Pts

- 也可以先令 $t_i \leftarrow \min(t_i, n)$ ，再将香料按 t_i 从小到大排序。处理到第 i 份香料时，小根堆 H 中保存的是目前暂时选中的香料收益，并且这些香料已经能够合法地安排在前 t_i 日。

正解 100 Pts

- 也可以先令 $t_i \leftarrow \min(t_i, n)$ ，再将香料按 t_i 从小到大排序。处理到第 i 份香料时，小根堆 H 中保存的是目前暂时选中的香料收益，并且这些香料已经能够合法地安排在前 t_i 日。
- 将当前收益加入 H 。若此时 $|H| > t_i$ ，根据前面的判定条件，这些香料不可能全部在失效前卖出，因此必须删去一份。删除堆顶，也就是收益最小的一份，能够在恢复可行性的同时使损失最小。

正解 100 Pts

- ▶ 也可以先令 $t_i \leftarrow \min(t_i, n)$ ，再将香料按 t_i 从小到大排序。处理到第 i 份香料时，小根堆 H 中保存的是目前暂时选中的香料收益，并且这些香料已经能够合法地安排在前 t_i 日。
- ▶ 将当前收益加入 H 。若此时 $|H| > t_i$ ，根据前面的判定条件，这些香料不可能全部在失效前卖出，因此必须删去一份。删除堆顶，也就是收益最小的一份，能够在恢复可行性的同时使损失最小。
- ▶ 之前已经满足的期限限制不会因为删除香料而被破坏。于是归纳可知，处理完每一个相同期限的前缀后， H 都保留了这个前缀中总收益最大的可行选择。

正解 100 Pts

- 对于一次固定的询问 p ，只需把上面的限制改成 $|H| \leq \min(t_i, p)$ 。每次超过限制时仍然删除当前最小收益，最终堆中的元素之和就是这次询问的答案。

正解 100 Pts

- ▶ 对于一次固定的询问 p ，只需把上面的限制改成 $|H| \leq \min(t_i, p)$ 。每次超过限制时仍然删除当前最小收益，最终堆中的元素之和就是这次询问的答案。
- ▶ 这些删除操作始终只会删去当前最小值，因此限制 $|H| \leq p$ 与最后只保留 H 中最大的 p 个收益是等价的。设完整过程最终保留 c 份香料，将其收益降序排列为 $b_1, b_2, \dots, b_c$ ，则记录 $\text{Ans}_s = \sum_{i=1}^s b_i$ 即可回答所有询问。

正解 100 Pts

- ▶ 对于一次固定的询问 p ，只需把上面的限制改成 $|H| \leq \min(t_i, p)$ 。每次超过限制时仍然删除当前最小收益，最终堆中的元素之和就是这次询问的答案。
- ▶ 这些删除操作始终只会删去当前最小值，因此限制 $|H| \leq p$ 与最后只保留 H 中最大的 p 个收益是等价的。设完整过程最终保留 c 份香料，将其收益降序排列为 $b_1, b_2, \dots, b_c$ ，则记录 $\text{Ans}_s = \sum_{i=1}^s b_i$ 即可回答所有询问。
- ▶ 对于询问 p ，输出 $\text{Ans}_{\min(p, c)}$ 。排序和堆维护的时间复杂度为 $\mathcal{O}(n \log n + q)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

- 另一种写法是把香料按照 t_i 分组，再依次处理 $d = n, n-1, \dots, 1$ 。开始处理第 d 日时，将所有满足 $t_i = d$ 的香料收益加入大根堆，此时堆中恰好保存了尚未安排且失效时间不早于 d 的香料。

正解 100 Pts

- ▶ 另一种写法是把香料按照 t_i 分组，再依次处理 $d = n, n-1, \dots, 1$ 。开始处理第 d 日时，将所有满足 $t_i = d$ 的香料收益加入大根堆，此时堆中恰好保存了尚未安排且失效时间不早于 d 的香料。
- ▶ 因而堆中的任意香料都能安排在第 d 日。若堆非空，就取出收益最大的一份放在这一天。考虑任意最优方案：若它在第 d 日安排了另一份收益更小的香料，就把两者交换；若收益最大的香料原本没有被选择，就直接用它替换第 d 日的香料，方案仍然合法且不会变劣。

正解 100 Pts

- ▶ 另一种写法是把香料按照 t_i 分组，再依次处理 $d = n, n-1, \dots, 1$ 。开始处理第 d 日时，将所有满足 $t_i = d$ 的香料收益加入大根堆，此时堆中恰好保存了尚未安排且失效时间不早于 d 的香料。
- ▶ 因而堆中的任意香料都能安排在第 d 日。若堆非空，就取出收益最大的一份放在这一天。考虑任意最优方案：若它在第 d 日安排了另一份收益更小的香料，就把两者交换；若收益最大的香料原本没有被选择，就直接用它替换第 d 日的香料，方案仍然合法且不会变劣。
- ▶ 依次确定每一天后，所有取出的香料自然构成一个合法方案，上述交换也说明这个方案的总收益最大。

正解 100 Pts

- ▶ 还需要说明为什么同一个方案能够回答所有询问。固定一个收益下界 W ，暂时只看收益不小于 W 的香料。倒序处理日期时，只要堆中存在这样的香料，大根堆就一定优先取出其中一份，因此该过程选中了尽可能多的收益不小于 W 的香料。

正解 100 Pts

- ▶ 还需要说明为什么同一个方案能够回答所有询问。固定一个收益下界 W ，暂时只看收益不小于 W 的香料。倒序处理日期时，只要堆中存在这样的香料，大根堆就一定优先取出其中一份，因此该过程选中了尽可能多的收益不小于 W 的香料。
- ▶ 将最终取出的收益降序排列为 $b_1, b_2, \dots, b_c$ 。若某个可行的 p 元方案中第 j 大的收益严格大于 b_j ，取这个收益作为 W ，便会得到一个包含更多高收益香料的可行方案，与上一条结论矛盾。因此前 p 项之和就是最多选择 p 份时的答案。

正解 100 Pts

- ▶ 还需要说明为什么同一个方案能够回答所有询问。固定一个收益下界 W ，暂时只看收益不小于 W 的香料。倒序处理日期时，只要堆中存在这样的香料，大根堆就一定优先取出其中一份，因此该过程选中了尽可能多的收益不小于 W 的香料。
- ▶ 将最终取出的收益降序排列为 $b_1, b_2, \dots, b_c$ 。若某个可行的 p 元方案中第 j 大的收益严格大于 b_j ，取这个收益作为 W ，便会得到一个包含更多高收益香料的可行方案，与上一条结论矛盾。因此前 p 项之和就是最多选择 p 份时的答案。
- ▶ 求出 b_i 的前缀和后即可 $\mathcal{O}(1)$ 回答询问。总时间复杂度为 $\mathcal{O}(n \log n + q)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

吐槽时间

数据点 1 ~ 3 15 Pts

- ▶ 先不考虑数据范围，把“当前位于岛屿 x ，灵舵档位为 p ”看成状态 (x, p) 。

数据点 1 ~ 3 15 Pts

- ▶ 先不考虑数据范围，把“当前位于岛屿 x ，灵舵档位为 p ”看成状态 (x, p) 。
- ▶ 调高、调低档位以及选择第 p 条航道，分别对应三类有向边。因为所有费用非负，从 $(1, 1)$ 运行 Dijkstra 即可。

数据点 1 ~ 3 15 Pts

- ▶ 先不考虑数据范围，把“当前位于岛屿 x ，灵舵档位为 p ”看成状态 (x, p) 。
- ▶ 调高、调低档位以及选择第 p 条航道，分别对应三类有向边。因为所有费用非负，从 $(1, 1)$ 运行 Dijkstra 即可。
- ▶ 这一档至多只有 $nk \leq 36$ 个状态，可以显式建立完整状态图。这个状态会在后面的每一档中继续使用。

数据点 4 ~ 6 30 Pts

- 仍然沿用状态 (x, p) ，总状态数为 nk 。每个状态至多产生两次调档转移，并在第 p 条航道存在时产生一次航行转移。

数据点 4 ~ 6 30 Pts

- ▶ 仍然沿用状态 (x, p) ，总状态数为 nk 。每个状态至多产生两次调档转移，并在第 p 条航道存在时产生一次航行转移。
- ▶ 调档边可以在访问状态时直接计算，不必真正存入邻接表。时间复杂度为 $\mathcal{O}((nk + m) \log(nk))$ ，空间复杂度为 $\mathcal{O}(nk + m)$ 。

数据点 4 ~ 6 30 Pts

- ▶ 仍然沿用状态 (x, p) ，总状态数为 nk 。每个状态至多产生两次调档转移，并在第 p 条航道存在时产生一次航行转移。
- ▶ 调档边可以在访问状态时直接计算，不必真正存入邻接表。时间复杂度为 $\mathcal{O}((nk + m) \log(nk))$ ，空间复杂度为 $\mathcal{O}(nk + m)$ 。
- ▶ 瓶颈在于：即使岛屿 x 只有 d_x 条出航航道，也为它保存了全部 k 个档位。接下来需要减少这些没有对应航道的状态。

数据点 7 ~ 9 45 Pts

- 当 $k \leq 100$ 时，完整状态图仍然能够保留。用 $(x-1)k+p$ 直接计算状态编号，并在弹出状态时计算相邻档位即可。

数据点 7 ~ 9 45 Pts

- ▶ 当 $k \leq 100$ 时, 完整状态图仍然能够保留。用 $(x-1)k+p$ 直接计算状态编号, 并在弹出状态时计算相邻档位即可。
- ▶ 算法与上一档相同, 时间复杂度仍为 $\mathcal{O}((nk+m)\log(nk))$ 。

数据点 7 ~ 9 45 Pts

- ▶ 当 $k \leq 100$ 时, 完整状态图仍然能够保留。用 $(x-1)k+p$ 直接计算状态编号, 并在弹出状态时计算相邻档位即可。
- ▶ 算法与上一档相同, 时间复杂度仍为 $\mathcal{O}((nk+m)\log(nk))$ 。
- ▶ 回顾这个做法可以发现, 档位 $p > d_x$ 时不能从岛屿 x 出航; 它的作用只可能是继续调档。这正是状态可以压缩的地方。

数据点 10 ~ 12 60 Pts

- ▶ 特殊性质 A 中，所有调档费用均为 0。到达任意岛屿后，可以免费切换到任意存在出航航道的档位。

数据点 10 ~ 12 60 Pts

- ▶ 特殊性质 A 中，所有调档费用均为 0。到达任意岛屿后，可以免费切换到任意存在出航航道的档位。
- ▶ 因此问题退化为原图上的单源最短路，边权就是航行费用，直接从岛屿 1 运行 Dijkstra。

数据点 10 ~ 12 60 Pts

- ▶ 特殊性质 A 中，所有调档费用均为 0。到达任意岛屿后，可以免费切换到任意存在出航航道的档位。
- ▶ 因此问题退化为原图上的单源最短路，边权就是航行费用，直接从岛屿 1 运行 Dijkstra。
- ▶ 时间复杂度为 $\mathcal{O}((n+m)\log n)$ ，空间复杂度为 $\mathcal{O}(n+m)$ 。这一档说明，真正需要记录的是调档对后续出航选择造成的影响。

数据点 13 ~ 15 75 Pts

- ▶ 特殊性质 B 中，至多有 10 座岛屿满足 $d_x \geq 10$ 。对这些岛屿保留全部 k 个档位，其余岛屿只保留 $1 \sim \max(1, d_x)$ 。

数据点 13 ~ 15 75 Pts

- ▶ 特殊性质 B 中，至多有 10 座岛屿满足 $d_x \geq 10$ 。对这些岛屿保留全部 k 个档位，其余岛屿只保留 $1 \sim \max(1, d_x)$ 。
- ▶ 两类岛屿分别产生至多 $10k$ 与 $9n$ 个状态，因此总状态数为 $\mathcal{O}(n + k)$ ，仍然可以运行最短路。

数据点 13 ~ 15 75 Pts

- ▶ 特殊性质 B 中，至多有 10 座岛屿满足 $d_x \geq 10$ 。对这些岛屿保留全部 k 个档位，其余岛屿只保留 $1 \sim \max(1, d_x)$ 。
- ▶ 两类岛屿分别产生至多 $10k$ 与 $9n$ 个状态，因此总状态数为 $\mathcal{O}(n + k)$ ，仍然可以运行最短路。
- ▶ 这一档已经给出了正解的压缩方向。剩下的问题是：到达岛屿 y 时，若当前档位 $p > d_y$ ，怎样既删去高档位状态，又保留“刚刚抵达 y ”的答案。

正解 100 Pts

► 为了快速计算连续降档费用，定义

$$\text{Pre}(1) = 0, \quad \text{Pre}(p) = \sum_{i=2}^p w_i \quad (p \geq 2).$$

于是从档位 p 连续降到 $q < p$ 的费用为 $\text{Pre}(p) - \text{Pre}(q)$ 。

正解 100 Pts

- 为了快速计算连续降档费用，定义

$$\text{Pre}(1) = 0, \quad \text{Pre}(p) = \sum_{i=2}^p w_i \quad (p \geq 2).$$

于是从档位 p 连续降到 $q < p$ 的费用为 $\text{Pre}(p) - \text{Pre}(q)$ 。

- 为了统一处理零出度岛屿，我们为每座岛屿 x 建立占位状态 $\text{Mark}(x, 0)$ ，再建立 $\text{Mark}(x, 1), \dots, \text{Mark}(x, d_x)$ 。

正解 100 Pts

- 为了快速计算连续降档费用，定义

$$\text{Pre}(1) = 0, \quad \text{Pre}(p) = \sum_{i=2}^p w_i \quad (p \geq 2).$$

于是从档位 p 连续降到 $q < p$ 的费用为 $\text{Pre}(p) - \text{Pre}(q)$ 。

- 为了统一处理零出度岛屿，我们为每座岛屿 x 建立占位状态 $\text{Mark}(x, 0)$ ，再建立 $\text{Mark}(x, 1), \dots, \text{Mark}(x, d_x)$ 。
- 当 $d_x > 0$ 时，只有后 d_x 个状态表示真实档位；当 $d_x = 0$ 时，占位状态表示已经抵达这座无法出航的岛屿。

正解 100 Pts

► 对 $2 \leq p \leq d_x$, 加入

$$\begin{aligned}\text{Mark}(x, p-1) &\xrightarrow{v_{p-1}} \text{Mark}(x, p), \\ \text{Mark}(x, p) &\xrightarrow{w_p} \text{Mark}(x, p-1).\end{aligned}$$

正解 100 Pts

- 对 $2 \leq p \leq d_x$, 加入

$$\begin{aligned}\text{Mark}(x, p-1) &\xrightarrow{v_{p-1}} \text{Mark}(x, p), \\ \text{Mark}(x, p) &\xrightarrow{w_p} \text{Mark}(x, p-1).\end{aligned}$$

- 设岛屿 x 的第 p 条航道通向 y , 航行费用为 z 。若 $p \leq d_y$, 直接连边 $\text{Mark}(x, p) \rightarrow \text{Mark}(y, p)$, 边权为 z 。

正解 100 Pts

- 对 $2 \leq p \leq d_x$, 加入

$$\begin{aligned}\text{Mark}(x, p-1) &\xrightarrow{v_{p-1}} \text{Mark}(x, p), \\ \text{Mark}(x, p) &\xrightarrow{w_p} \text{Mark}(x, p-1).\end{aligned}$$

- 设岛屿 x 的第 p 条航道通向 y , 航行费用为 z 。若 $p \leq d_y$, 直接连边 $\text{Mark}(x, p) \rightarrow \text{Mark}(y, p)$, 边权为 z 。
- 若 $d_y = 0$, 则连向 $\text{Mark}(y, 0)$ 。目标岛屿没有出航航道, 因此不需要保留到达时的具体档位。

正解 100 Pts

- 最后考虑 $0 < d_y < p$ 。为了同时保留“刚抵达 y ”的费用与之后的降档过程，我们为这次航行定义中转状态 t ，令 $\text{Back}(t) = y$ ，并加入

$$\text{Mark}(x, p) \xrightarrow{z} t \xrightarrow{\text{Pre}(p) - \text{Pre}(d_y)} \text{Mark}(y, d_y).$$

正解 100 Pts

- ▶ 最后考虑 $0 < d_y < p$ 。为了同时保留“刚抵达 y ”的费用与之后的降档过程，我们为这次航行定义中转状态 t ，令 $\text{Back}(t) = y$ ，并加入

$$\text{Mark}(x, p) \xrightarrow{z} t \xrightarrow{\text{Pre}(p) - \text{Pre}(d_y)} \text{Mark}(y, d_y).$$

- ▶ 状态 t 表示“已经以档位 p 抵达岛屿 y ，但尚未进行之后的调档”。因此 t 的距离就是这次抵达 y 的真实费用。

正解 100 Pts

- 最后考虑 $0 < d_y < p$ 。为了同时保留“刚抵达 y ”的费用与之后的降档过程，我们为这次航行定义中转状态 t ，令 $\text{Back}(t) = y$ ，并加入

$$\text{Mark}(x, p) \xrightarrow{z} t \xrightarrow{\text{Pre}(p) - \text{Pre}(d_y)} \text{Mark}(y, d_y).$$

- 状态 t 表示“已经以档位 p 抵达岛屿 y ，但尚未进行之后的调档”。因此 t 的距离就是这次抵达 y 的真实费用。
- 对每个状态 s 记录它所属的岛屿 $\text{Back}(s)$ ，最终有

$$\text{Ans}(x) = \min_{\text{Back}(s)=x} \text{Dist}(s).$$

这一步同时统计普通状态、零出度占位状态与中转状态。

正解 100 Pts

- ▶ 下面证明压缩不会改变答案。若在完整状态图中以档位 $p > d_y$ 抵达 y ，下一次从 y 出航前一定要降到某个 $q \leq d_y$ 。

正解 100 Pts

- 下面证明压缩不会改变答案。若在完整状态图中以档位 $p > d_y$ 抵达 y ，下一次从 y 出航前一定要降到某个 $q \leq d_y$ 。
- 删除这期间所有往返调档不会增加费用；剩余的连续下降必然先经过 d_y ，并且费用可以拆成

$$(\text{Pre}(p) - \text{Pre}(d_y)) + (\text{Pre}(d_y) - \text{Pre}(q)).$$

所以中转状态接到 $\text{Mark}(y, d_y)$ 不会损失最优的后续方案。

正解 100 Pts

- 下面证明压缩不会改变答案。若在完整状态图中以档位 $p > d_y$ 抵达 y ，下一次从 y 出航前一定要降到某个 $q \leq d_y$ 。
- 删除这期间所有往返调档不会增加费用；剩余的连续下降必然先经过 d_y ，并且费用可以拆成

$$(\text{Pre}(p) - \text{Pre}(d_y)) + (\text{Pre}(d_y) - \text{Pre}(q)).$$

所以中转状态接到 $\text{Mark}(y, d_y)$ 不会损失最优的后续方案。

- 反过来，中转边可以展开为从 p 到 d_y 的逐次降档，其余边本来就是一次合法操作。因此压缩图中的每条路径都能还原到完整状态图，两个方向的最短费用相同。

正解 100 Pts

- 固定状态共有 $n + \sum_x d_x = n + m$ 个；每条航道至多额外产生一个中转状态，所以总状态数不超过 $n + 2m$ 。

正解 100 Pts

- ▶ 固定状态共有 $n + \sum_x d_x = n + m$ 个；每条航道至多额外产生一个中转状态，所以总状态数不超过 $n + 2m$ 。
- ▶ 相邻档位边至多 $2m$ 条，航行边恰有 m 条，中转状态的降档边至多 m 条，因此总边数为 $\mathcal{O}(n + m)$ 。

正解 100 Pts

- ▶ 固定状态共有 $n + \sum_x d_x = n + m$ 个；每条航道至多额外产生一个中转状态，所以总状态数不超过 $n + 2m$ 。
- ▶ 相邻档位边至多 $2m$ 条，航行边恰有 m 条，中转状态的降档边至多 m 条，因此总边数为 $\mathcal{O}(n + m)$ 。
- ▶ 若 $d_1 > 0$ ，起点为 $\text{Mark}(1, 1)$ ；否则起点为 $\text{Mark}(1, 0)$ 。运行 Dijkstra 的时间复杂度为 $\mathcal{O}((n + m) \log(n + m))$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。

正解 100 Pts

- ▶ 固定状态共有 $n + \sum_x d_x = n + m$ 个；每条航道至多额外产生一个中转状态，所以总状态数不超过 $n + 2m$ 。
- ▶ 相邻档位边至多 $2m$ 条，航行边恰有 m 条，中转状态的降档边至多 m 条，因此总边数为 $\mathcal{O}(n + m)$ 。
- ▶ 若 $d_1 > 0$ ，起点为 $\text{Mark}(1, 1)$ ；否则起点为 $\text{Mark}(1, 0)$ 。运行 Dijkstra 的时间复杂度为 $\mathcal{O}((n + m) \log(n + m))$ ，空间复杂度为 $\mathcal{O}(n + m)$ 。
- ▶ 距离与前缀费用必须使用 long long；不可达岛屿输出 -1 。

正解 100 Pts

```
1  Pre[1]=0;
2  scanf("%d%d%d",&n,&m,&k);
3  for(int i=1;i<=k-1;i++){
4      scanf("%lld",&UpValue[i]);
5  }
6  for(int i=2;i<=k;i++){
7      scanf("%lld",&DownValue[i]);
8      Pre[i]=DownValue[i]+Pre[i-1];
9  }
```

正解 100 Pts

```
1  for(int i=1;i<=n;i++){
2      scanf("%d",&Deg[i]);
3      cnt++;
4      Mark[i].push_back(cnt);
5      Back[cnt]=i;
6      for(int j=1;j<=Deg[i];j++){
7          cnt++;
8          Mark[i].push_back(cnt);
9          Back[cnt]=i;
10         if(j!=1){
11             Edge_Add(Mark[i][j],Mark[i][j-1],DownValue[j]);
12             Edge_Add(Mark[i][j-1],Mark[i][j],UpValue[j-1]);
13         }
14     }
```

正解 100 Pts

```
1      for(int j=1;j<=Deg[i];j++){
2          cne++;
3          scanf("%d%d",&Ed[cne],&EdgeV[cne]);
4          Tag[cne]=j;St[cne]=i;
5      }
6  }
```

正解 100 Pts

```
1  S=Mark[1][Deg[1]?1:0];
2  for(int i=1;i<=m;i++){
3      if(!Deg[Ed[i]]){
4          Edge_Add(Mark[St[i]][Tag[i]],Mark[Ed[i]][0],EdgeV[i]);
5      }
6      else if(Deg[Ed[i]]<Tag[i]){
7          cnt++;Mark[Ed[i]].push_back(cnt);Back[cnt]=Ed[i];
8          Edge_Add(Mark[St[i]][Tag[i]],cnt,EdgeV[i]);
9          Edge_Add(cnt,Mark[Ed[i]][Deg[Ed[i]]],
10                  Pre[Tag[i]]-Pre[Deg[Ed[i]]]);
11      }
12      else{
13          Edge_Add(Mark[St[i]][Tag[i]],Mark[Ed[i]][Tag[i]],EdgeV[i]);
14      }
15  }
```

正解 100 Pts

```
1  Dijkstra();
2  Ans[1]=0;
3  for(int i=1;i<=cnt;i++){
4      Ans[Back[i]]=min(Ans[Back[i]],Dist[i]);
5  }
6  for(int i=1;i<=n;i++){
7      if(Ans[i]==0x3f3f3f3f3f3f3f3f){
8          printf("-1 ");
9      }
10     else printf("%lld ",Ans[i]);
11 }
12 return 0;
```


数据点 1 ~ 3 15 Pts

- ▶ 首先枚举遗迹入口，再枚举遗室的清理顺序以及每间新遗室连接到哪一间已经清理的遗室。
- ▶ 对每个方案递推出所有探索层数，并直接计算总代价，取最小值即可。

数据点 1 ~ 3 15 Pts

- ▶ 首先枚举遗迹入口，再枚举遗室的清理顺序以及每间新遗室连接到哪一间已经清理的遗室。
- ▶ 对每个方案递推出所有探索层数，并直接计算总代价，取最小值即可。
- ▶ 这个暴力完整枚举了入口、父亲与层数，但是顺序和父亲的选择都会产生指数级复杂度。接下来需要把只与已清理集合有关的方案合并起来。

数据点 4 ~ 6 30 Pts

- 这一档中原图是一棵树。固定遗迹入口 r 后，每个遗室的父亲与探索层数都被唯一确定。

数据点 4 ~ 6 30 Pts

- ▶ 这一档中原图是一棵树。固定遗迹入口 r 后，每个遗室的父亲与探索层数都被唯一确定。
- ▶ 一次 DFS 求出深度后，总代价为

$$\sum_{v \neq r} \text{dep}_r(v) \cdot w(v, \text{fa}_r(v)).$$

数据点 4 ~ 6 30 Pts

- ▶ 这一档中原图是一棵树。固定遗迹入口 r 后，每个遗室的父亲与探索层数都被唯一确定。
- ▶ 一次 DFS 求出深度后，总代价为

$$\sum_{v \neq r} \text{dep}_r(v) \cdot w(v, \text{fa}_r(v)).$$

- ▶ 枚举入口并分别 DFS，时间复杂度为 $\mathcal{O}(n^2)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 7 ~ 10 50 Pts

► 对遗室 v 与非空集合 S , 定义

$$C(v, S) = \min_{u \in S} w(u, v),$$

若不存在这样的通道, 则令 $C(v, S) = +\infty$ 。

数据点 7 ~ 10 50 Pts

- 对遗室 v 与非空集合 S , 定义

$$C(v, S) = \min_{u \in S} w(u, v),$$

若不存在这样的通道, 则令 $C(v, S) = +\infty$ 。

- 对 $P \subseteq S$, 再定义

$$G(P, S) = \sum_{v \in S \setminus P} C(v, P).$$

只有当每个 $v \in S \setminus P$ 都能与 P 相连时, 这个转移才合法。

数据点 7 ~ 10 50 Pts

- 考虑一条集合扩展链 $\{r\} = S_1 \subsetneq S_2 \subsetneq \cdots \subsetneq S_d = S$ ，并把第 i 次扩展的费用记为 $(i-1)G(S_{i-1}, S_i)$ 。

数据点 7 ~ 10 50 Pts

- ▶ 考虑一条集合扩展链 $\{r\} = S_1 \subsetneq S_2 \subsetneq \cdots \subsetneq S_d = S$, 并把第 i 次扩展的费用记为 $(i-1)G(S_{i-1}, S_i)$ 。
- ▶ 定义 $f_{d,S}$ 表示所有以 S 结束、长度为 d 的集合扩展链中, 上述费用之和的最小值。对每个入口 r , 初始化

$$f_{1,\{r\}} = 0,$$

其余状态均为 $+\infty$ 。

数据点 7 ~ 10 50 Pts

- ▶ 考虑一条集合扩展链 $\{r\} = S_1 \subsetneq S_2 \subsetneq \dots \subsetneq S_d = S$ ，并把第 i 次扩展的费用记为 $(i-1)G(S_{i-1}, S_i)$ 。
- ▶ 定义 $f_{d,S}$ 表示所有以 S 结束、长度为 d 的集合扩展链中，上述费用之和的最小值。对每个入口 r ，初始化

$$f_{1,\{r\}} = 0,$$

其余状态均为 $+\infty$ 。

- ▶ 每次必须加入至少一间新遗室，因此完整转移为

$$f_{d,S} = \min_{\emptyset \neq P \subsetneq S} \{f_{d-1,P} + (d-1)G(P, S)\}.$$

不存在相应通道或前驱不可行时，这一项视为 $+\infty$ 。

数据点 7 ~ 10 50 Pts

- 转移顺序为 $d = 2, 3, \dots, n$ ，对每个 d 枚举全部集合 S ，再枚举 S 的非空真子集 P 。

数据点 7 ~ 10 50 Pts

- ▶ 转移顺序为 $d = 2, 3, \dots, n$ ，对每个 d 枚举全部集合 S ，再枚举 S 的非空真子集 P 。
- ▶ 最终答案为

$$\min_{1 \leq d \leq n} f_{d, \{1, 2, \dots, n\}}.$$

数据点 7 ~ 10 50 Pts

- ▶ 转移顺序为 $d = 2, 3, \dots, n$ ，对每个 d 枚举全部集合 S ，再枚举 S 的非空真子集 P 。
- ▶ 最终答案为

$$\min_{1 \leq d \leq n} f_{d, \{1, 2, \dots, n\}}.$$

- ▶ 直接计算每次转移中的 $G(P, S)$ 需要再枚举一遍 $S \setminus P$ ，这一档规模较小，可以直接实现。

数据点 11 ~ 15 75 Pts

- 先预处理所有 $C(v, S)$ 。取出 S 中任意一个元素 u ，就有

$$C(v, S) = \min(C(v, S \setminus \{u\}), w(u, v)).$$

时间复杂度为 $\mathcal{O}(n2^n)$ 。

数据点 11 ~ 15 75 Pts

- 先预处理所有 $C(v, S)$ 。取出 S 中任意一个元素 u ，就有

$$C(v, S) = \min(C(v, S \setminus \{u\}), w(u, v)).$$

时间复杂度为 $\mathcal{O}(n2^n)$ 。

- 再对每一对 $P \subsetneq S$ 预处理 $G(P, S)$ ，并删去其中含有 $C(v, P) = +\infty$ 的转移。之后每次 DP 转移只需要 $\mathcal{O}(1)$ 。

数据点 11 ~ 15 75 Pts

- ▶ 先预处理所有 $C(v, S)$ 。取出 S 中任意一个元素 u ，就有

$$C(v, S) = \min(C(v, S \setminus \{u\}), w(u, v)).$$

时间复杂度为 $\mathcal{O}(n2^n)$ 。

- ▶ 再对每一对 $P \subsetneq S$ 预处理 $G(P, S)$ ，并删去其中含有 $C(v, P) = +\infty$ 的转移。之后每次 DP 转移只需要 $\mathcal{O}(1)$ 。
- ▶ 子集对数量为 $\mathcal{O}(3^n)$ ，这一档已经得到了正解的全部状态和转移，只剩下严格说明它与真实探索层数的关系。

正解 100 Pts

- ▶ 先从任意一条 DP 转移构造探索方案。对每个 $v \in S \setminus P$, 选择一条达到 $C(v, P)$ 的通道, 把 v 接到对应的已清理遗室。

正解 100 Pts

- ▶ 先从任意一条 DP 转移构造探索方案。对每个 $v \in S \setminus P$, 选择一条达到 $C(v, P)$ 的通道, 把 v 接到对应的已清理遗室。
- ▶ 这个父亲可能位于第 $d - 2$ 层以前, 所以 v 的真实层数至多为 $d - 1$ 。因此它产生的真实开辟代价不超过 DP 中计算的 $(d - 1)C(v, P)$ 。

正解 100 Pts

- ▶ 先从任意一条 DP 转移构造探索方案。对每个 $v \in S \setminus P$, 选择一条达到 $C(v, P)$ 的通道, 把 v 接到对应的已清理遗室。
- ▶ 这个父亲可能位于第 $d - 2$ 层以前, 所以 v 的真实层数至多为 $d - 1$ 。因此它产生的真实开辟代价不超过 DP 中计算的 $(d - 1)C(v, P)$ 。
- ▶ 逐次转移便能构造一个合法方案, 其真实总代价不超过对应 DP 值。于是最优探索代价不大于 DP 的答案。

正解 100 Pts

- 再取任意一个最优探索方案，按照真实层数分组。令 S_d 表示探索层数不超过 $d - 1$ 的遗室集合，于是

$$S_1 \subsetneq S_2 \subsetneq \cdots \subseteq \{1, 2, \dots, n\}.$$

正解 100 Pts

- ▶ 再取任意一个最优探索方案，按照真实层数分组。令 S_d 表示探索层数不超过 $d-1$ 的遗室集合，于是

$$S_1 \subsetneq S_2 \subsetneq \cdots \subseteq \{1, 2, \dots, n\}.$$

- ▶ 对每个 $v \in S_d \setminus S_{d-1}$ ，它的父亲属于 S_{d-1} ，所以原方案使用的边权不小于 $C(v, S_{d-1})$ 。

正解 100 Pts

- ▶ 再取任意一个最优探索方案，按照真实层数分组。令 S_d 表示探索层数不超过 $d-1$ 的遗室集合，于是

$$S_1 \subsetneq S_2 \subsetneq \cdots \subseteq \{1, 2, \dots, n\}.$$

- ▶ 对每个 $v \in S_d \setminus S_{d-1}$ ，它的父亲属于 S_{d-1} ，所以原方案使用的边权不小于 $C(v, S_{d-1})$ 。
- ▶ 因而 DP 取 $P = S_{d-1}$ 、 $S = S_d$ 时的这一层贡献不大于原方案的对应贡献。把所有层相加，DP 的答案不大于最优探索代价。

正解 100 Pts

- ▶ 上两页分别得到“最优探索代价不大于 DP 答案”与“DP 答案不大于最优探索代价”，所以两者相等。

正解 100 Pts

- ▶ 上两页分别得到“最优探索代价不大于 DP 答案”与“DP 答案不大于最优探索代价”，所以两者相等。
- ▶ 状态数为 $\mathcal{O}(n2^n)$ ，合法子集对总数为 $\mathcal{O}(3^n)$ ；标准程序按层枚举，因此时间复杂度为 $\mathcal{O}(n3^n)$ 。

正解 100 Pts

- ▶ 上两页分别得到“最优探索代价不大于 DP 答案”与“DP 答案不大于最优探索代价”，所以两者相等。
- ▶ 状态数为 $\mathcal{O}(n2^n)$ ，合法子集对总数为 $\mathcal{O}(3^n)$ ；标准程序按层枚举，因此时间复杂度为 $\mathcal{O}(n3^n)$ 。
- ▶ 空间复杂度为 $\mathcal{O}(n2^n + 3^n)$ 。重边只保留最小边权，DP 与代价数组均使用 long long，非法状态用足够大的正无穷表示。

正解 100 Pts

```
1  for(int i=0;i<(1<<n);i++){
2      for(int j=1;j<=n;j++){
3          if((i&(1<<(j-1)))==0) continue;
4          for(int k=1;k<=n;k++){
5              Cost[k][i]=min(Cost[k][i],1ll*Val[k][j]);
6          }
7      }
8  }
```

正解 100 Pts

```
1  for(int i=0;i<(1<<n);i++){
2      for(int j=(i-1)&i;;j=(j-1)&i){
3          int Diff=i-j; bool Tag=1;
4          long long Sum=0;
5          while(Diff){
6              if(Cost[B[Lowbit(Diff)]] [j]>1e6){Tag=0; break;}
7              Sum+=Cost[B[Lowbit(Diff)]] [j];
8              Diff-=Lowbit(Diff);
9          }
10         if(Tag){Valid[i].push_back(j); Pri[i].push_back(Sum);}
11         if(j==0) break;
12     }
13 }
```

正解 100 Pts

```
1  for (int i=1;i<=n;i++) {
2      Dp[1][1<<(i-1)]=0;
3  }
4  for(int i=1;i<=n;i++){
5      for(int j=0;j<=(1<<n)-1;j++){
6          for(int k=0;k<Valid[j].size();k++){
7              Dp[i][j]=min(Dp[i][j],Dp[i-1][Valid[j][k]]+(i-1)*Pri[j][
8              k]);
9          }
10         Ans=min(Ans,Dp[i][(1<<n)-1]);
11     }
```

Contents

T1 藿香正气液 (elixir)

T2 晨汐散余香 (aroma)

T3 灵舵渡千屿 (helm)

T4 沧溟探秘藏 (treasure)

吐槽时间